

Michael Fox

CMPE 172

29, April 2026

M6 Assignment

Logging Plan

Appointment Request System Events:

- Request receive
- Slot availability checked
- Slot reserved
- Appointment created
- Appointment cancelled, etc.

System Error Events:

- Slot is already booked
- Appointment request is invalid
- DB connection failure
- Transaction Rollback
- Time out

Security System Events:

- Login attempt success or failure
- App start/shutdown
- Health Check Request
- Unauthorized access attempt

Ex: log messages:

```
Info: {  
  "level": "INFO",  
  "event": "appointment_created",  
  "message": "Appointment successfully created",  
  "appointmentId": "A1024",  
  "patientId": "P456",  
  "providerId": "D12",
```

```
"correlationId": "req-789",  
"timestamp": "2026-04-29T10:30:00Z"  
}
```

```
Error: {  
  "level": "ERROR",  
  "event": "appointment_booking_failed",  
  "message": "Appointment booking failed because slot was already taken",  
  "slotId": "S900",  
  "reason": "slot_already_reserved",  
  "correlationId": "req-790",  
  "timestamp": "2026-04-29T10:32:00Z"  
}
```

```
Warning: {  
  "level": "WARN",  
  "event": "DB_Connection_Failure",  
  "message": "DB Connection response time exceeded threshold",  
  "service": "DB_Connection",  
  "latencyMs": 2800,  
  "correlationId": "req-789",  
  "timestamp": "2026-04-29T10:31:00Z"  
}
```

Monitoring Metric:

- Appointment Booking success rate → successful bookings/ total attempts
- Failed Bookings/Hour
- Avg Booking Latency → avg (time from appointment request received to appointment created)

Failure Scenario:

Double Booking an Appointment Time Slot → Race Condition

1. Pt Bob attempts to book timeslot 0800 with provider Jim

2. Pt Sam attempts to book the same time slot at about the same time
3. Both requests check availability
4. Pt Bob reserves the slot first
5. Pt Sam's attempt fails because the slot is no longer available

Recovery From Failure:

The booking operation will be taken care of with in a db transaction.

Code snippet within the Appointment Service Class:

```
@Transactional no usages 1 related problem
public Appointment bookAppointment(AppointmentRequest request) {
    TimeSlot slot = slotRepository.findById(request.getSlotId())
        .orElseThrow(() -> new RuntimeException("Slot not found"));

    if (!slot.isAvailable()) {
        logger.error("Appointment booking failed: slot already reserved. slotId={}", request.getSlotId());
        throw new RuntimeException("Slot already reserved");
    }

    slot.setAvailable(false);
    slotRepository.save(slot);

    Appointment appointment = appointmentRepository.save(new Appointment(request, slot));

    logger.info("Appointment successfully created. appointmentId={}", appointment.getId());

    return appointment;
}
```

The transaction is sent an appointment request, a timeslot is created locally with the requested timeSlotID. Then it checks to see if the slot is available, if not it will throw a runtime exception caught in the transaction and then rolled back because of failure, this prevents double bookings. In the case above Sam will receive a message notifying them that some else just booked the appointment.

Health Check:

This endpoint GET /health is used to evaluate if the application is running correctly. It will return a status report about the system. UP means its running, DOWN if app is not able to handle requests. DEGRADED if application is running but non key dependencies are failing.

EX: {

"status": "UP",

"timestamp": "2026-04-29T10:35:00Z",

"services": {

"application": "UP",

```
"database": "UP",  
}  
}
```

In the Controllers folder there will be a class: HealthController.java

```

13 @RestController
14 public class HealthController {
15
16     private final DataSource dataSource; 2 usages
17
18     public HealthController(DataSource dataSource) { no usages
19         this.dataSource = dataSource;
20     }
21
22     @GetMapping("/health")
23     public ResponseEntity<Map<String, Object>> getHealth() {
24         Map<String, Object> response = new HashMap<>();
25         Map<String, String> services = new HashMap<>();
26
27         boolean databaseUp = checkMySQLConnection();
28
29         services.put("application", "UP");
30         services.put("database", databaseUp ? "UP" : "DOWN");
31
32         response.put("status", databaseUp ? "UP" : "DOWN");
33         response.put("timestamp", Instant.now().toString());
34         response.put("services", services);
35
36         if (databaseUp) {
37             return ResponseEntity.ok(response);
38         } else {
39             return ResponseEntity
40                 .status(HttpStatus.SERVICE_UNAVAILABLE)
41                 .body(response);
42         }
43     }
44
45     private boolean checkMySQLConnection() { 1 usage
46         try (Connection connection = dataSource.getConnection()) {
47             return connection.isValid( timeout: 2);
48         } catch (Exception e) {
49             return false;
50         }
51     }
52 }

```

The datasource is the mySQL db. Spring boot gives the controller access to the connection configuration. The function will return a response body which will become a Json

similar to the one above. If this function is running, then the application is UP and the Boolean `databaseUp` will indicate if the DB is UP or DOWN.